

Modeling sequences of user actions for statistical goal recognition

**Marcelo G. Armentano & Analía
A. Amandi**

User Modeling and User-Adapted Interaction

The Journal of Personalization Research

ISSN 0924-1868

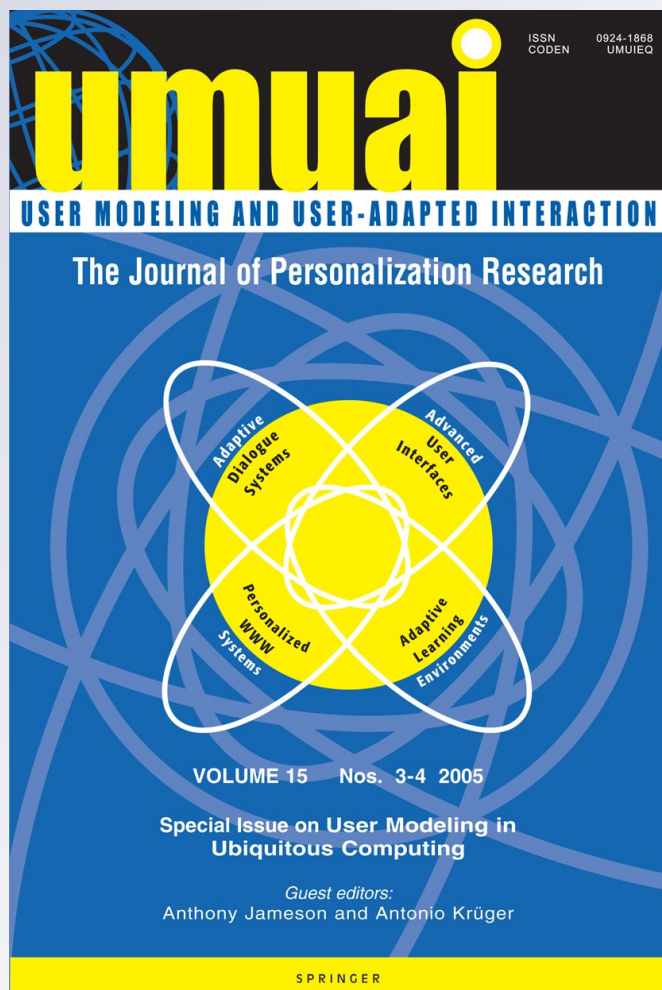
Volume 22

Number 3

User Model User-Adap Inter (2012)

22:281-311

DOI 10.1007/s11257-011-9103-y



Your article is protected by copyright and all rights are held exclusively by Springer Science+Business Media B.V.. This e-offprint is for personal use only and shall not be self-archived in electronic repositories. If you wish to self-archive your work, please use the accepted author's version for posting to your own website or your institution's repository. You may further deposit the accepted author's version on a funder's repository at a funder's request, provided it is not made publicly available until 12 months after publication.

Modeling sequences of user actions for statistical goal recognition

Marcelo G. Armentano · Analía A. Amandi

Received: 16 May 2010 / Accepted in revised form: 6 September 2011 /
Published online: 16 September 2011
© Springer Science+Business Media B.V. 2011

Abstract User goals are of major importance for an interface agent because they serve as a context to define what the user's focus of attention is at a given moment. The user's goals should be detected as soon as possible, after observing few user actions, in order to provide the user with timely assistance. In this article, we describe an approach for modeling and recognizing user goals from observed sequences of user actions by using Variable Order Markov models combined with an exponential moving average (EMA) on the prediction probabilities. The validity of our approach has been tested using data collected from real users in the Unix domain. The results obtained show that an interface agent can achieve near 90% average accuracy and over 58% online accuracy in predicting the most probable user goal after each observed action, in a time linear to the number of goals being modeled. We also found that the use of an EMA allows a faster convergence in the actual user goal.

Keywords Goal recognition · Variable Order Markov models · Interface agents · User modeling

M. G. Armentano (✉) · A. A. Amandi
Department of Computer Science, UNICEN University, Campus Universitario (B7001BBO),
Tandil, Argentina
e-mail: marmenta@exa.unicen.edu.ar

A. A. Amandi
e-mail: amandi@exa.unicen.edu.ar

M. G. Armentano · A. A. Amandi
ISISTAN Research Institute, Fac. Cs. Exactas, UNCPBA, Tandil, Argentina

M. G. Armentano · A. A. Amandi
CONICET, National Council of Scientific and Technological Research, Tandil, Argentina

1 Introduction

Users exhibit recurrent patterns of behavior when they interact with a software application. These recurrent patterns are determined by the goal the user has at a given moment (Card et al. 1983). It is essential to understand and model a user's behavior in order to help the user reach his/her goals, to predict the user future behavior and to coordinate communication.

The term that has been introduced to describe the process of inferring a subject's goals based on the actions he/she performs in an environment is *Goal Recognition*. In general, inputs to a goal recognition system are: (a) a set of goals the agent expects the user to achieve in the domain, (b) a set of plans describing the way in which the user can reach each goal, and (c) an action observed by the agent. The set of possible plans to achieve goals makes up a *plan library*. The goal recognition process itself consists in foretelling the user's goal, and determining how the observed action contributes to achieve this goal. Goal recognition is a special case of *plan recognition* in which the plan selected by the subject to achieve his/her goal is also detected. Although plan recognition is more informative than goal recognition, in many domains it is preferable to have a quick inference of the user goal rather than a slower inference of the complete user plan, which includes not only the user goals but also all the future actions the user will perform.

The work presented in this article is framed within interface agent technology. Interface agents (Maes 1994) are computer programs designed to assist human users in their computer-based tasks in a personalized manner. This kind of agent is able to learn interests, preferences, priorities, goals and needs of a user aiming at providing him/her with proactive and reactive assistance in order to increase the user's productivity regarding the application at issue.

A commonly used metaphor to understand the interface agent paradigm is comparing agents to a human secretary or personal assistant who collaborates with the user in the same work environment. Initially, a personal assistant is not very familiar with the habits and preferences of his/her boss and may not be very helpful. The personal assistant needs to learn about the particular work habits of his/her boss and gradually becomes more effective and competent as he/she acquires this knowledge. Then, the personal assistant can autonomously initiate an interaction by performing activities that were initially done by his/her boss, suggesting the execution of tasks, notifying interesting situations and giving warnings about problems that may arise.

With the aim of assisting a user of a software application, interface agents not only need to learn the user's preferences and habits regarding the use of the application itself, but they should also consider what his/her goal is before initiating an interaction with the user. The consideration of the status of a user's attention (i.e., the goal he/she is pursuing) and the uncertainty about the user's goals are critical factors for the effective integration of automated services with direct manipulation interfaces (Horvitz et al. 1998). A correct detection of the user's goals will prevent the agent from interrupting the user at an improper moment. Users generally do not want to be interrupted while working on a specific task, unless this interruption is closely related to the task they are performing (Whitworth 2005). By considering the user's current goals, the agent will be able to satisfy the user's requirements more effectively.

Since we focus on adding the capability of performing goal recognition to an interface agent, we consider goal recognition as the process of inferring a user's goals based on the observation of the actions he/she performs in a software application. Henceforward, we will use the term *user* to refer to the subject being observed, and the term *software application*, or just *application* to refer to the environment in which the subject is interacting.

An intuitive idea to perform plan recognition is to narrow the number of possible goals the agent believes the user is pursuing after each observed action. For example, when starting a scheduling application, the user can have any goal. However, if the agent observes that the user performs certain action, such as selecting "Add new contact" in the application menu, the set of goals is reduced to those in which the action performed is included as a particular step (for example, organizing an event with the new contact as a participant, or sending an email to him/her). If the next observed action is "Compose new email", the set of candidate goals can be further narrowed. Note that even if there is only one candidate goal, the agent will not always be absolutely sure that the user is pursuing that goal. In this way, each time the user performs an action in the application, the set of candidate goals might be reduced and/or some goal will be more probable than others.

In order to detect the user's current goal(s) the interface agent needs some representation describing the way those goals can be achieved in the application domain. This representation is commonly known as *plan library*. Traditionally, plan libraries have been hand-coded by a domain expert (Kautz 1991; Charniak and Goldman 1991; Horvitz et al. 1998; Lesh et al. 1999; Goldman et al. 1999; Pynadath and Wellman 2000; Geib et al. 2008). Manually designed plan libraries have the main advantage that can be highly expressive, facilitating the definition of arbitrary constraints between the steps in the plans and even logical relations between parameters. However, building a plan library is a tedious and error prone task and the success of a plan recognizer firstly relies on the correctness and completeness of the plan library itself. For this reason, in recent years researchers have paid special attention to the acquisition of plan libraries by constructing models that capture regularities in the user's behavior. Part of these research efforts was devoted to learning the parameters of the model used for recognition, such as probabilities, while the structure of the model itself remained fixed (Oliver et al. 2002; Bui 2003; Philipose et al. 2004; Nguyen et al. 2005; Pascal et al. 2005; Duong et al. 2006; Liao et al. 2007). However, in many situations the structure of the model cannot be given *a priori* and has to be learned from the interaction history of a user with a software application. Furthermore, behavior usually differs from one user to another and a predefined structure of plans may not fit a specific user behavior. For these reasons, many research groups have devoted their efforts to learn plan libraries by using machine learning techniques (Bauer 1998, 1999; Davison and Hirsh 1998; Gorniak and Poole 2000; Garland and Lesh 2002; Blaylock and Allen 2003; Madani et al. 2009; Nkambou et al. 2011).

Our work is framed within the last mentioned body of research: by using a well-known statistical machine learning technique, namely Variable Order Markov (VOM) models, we analyze the sequence of actions a user performs to achieve goals in a given application domain. We also propose the use of an exponential moving average (EMA) on the probabilities assigned by these models to the observed sequence

of actions performed by the user in order to predict the most likely goal the user is pursuing. We intend our model to

1. perform online recognition, i.e., to be able to make a prediction after every observed action (with an associated probability),
2. converge early, i.e., to be able to converge when not all the sequence of actions needed to achieve a given goal has been observed, and
3. be portable to new domains, i.e., little information about the domain in which it is applied is needed for the model to work.

The remainder of this paper is organized as follows: In Sect. 2 we describe related work and different techniques that can be used to address the problem. In Sect. 3 we present some background on VOM models and in Sect. 4 we present our adaptation of the VOMM theory to the behavior modeling domain and describe the learning algorithms we designed to learn these models from examples. In Sect. 5 we describe the experiments we performed and discuss the results obtained. Finally, in Sect. 6 we conclude and discuss future lines of research.

2 Related work

The prediction of a user's goal is an inherently uncertain task. Therefore, it is desirable to have a knowledge representation capable of dynamically capturing and modeling the uncertainty of human-computer interactions. Non-probabilistic approaches ([Kautz 1991](#); [Lesh 1998](#); [Rich et al. 2001](#); [Hong 2001](#)) are weakened by the fact that they cannot decide to what degree the evidence supports any particular hypothesis about the user's goal. This is an important issue to consider for the agent to be able to rank different possible explanations supported by the set of performed actions. Bayesian networks and Markov models are two of the most important representations for dealing with this kind of information.

Bayesian networks have been successfully used in previous approaches to plan recognition ([Charniak and Goldman 1991](#); [Horvitz et al. 1998](#); [Brown 1998](#); [Armentano and Amandi 2011](#)). Although good results can be achieved by using this representation, the structure of the network is usually hand-coded. There exist algorithms for learning both the structure [e.g., K2 ([Cooper and Herskovits 1992](#)), B ([Buntine 1991](#)) and PC ([Spirtes et al. 2000](#)) algorithms] and the parameters of a Bayesian network given a database of training examples [e.g., EM algorithm ([Dempster et al. 1977](#))]. A disadvantage of these approaches is that for these algorithms all the examples must be given with a fixed length, i.e., each example needs to be expressed with the same number of actions. However, this will not be the case when we observe a user's interaction with an application. We face the fact that the number of actions needed to accomplish different objectives may vary in a substantial way. Moreover, there may exist different ways to achieve the same goal, each of which may consist of a different number of steps (i.e., a different number of actions that need to be performed to achieve the goal).

This problem can be solved by using a sliding window of fixed size to segment the interaction history, or by considering that each example has the vocabulary size, that is, the complete number of possible actions the user can perform in the application. However, the first approach creates fictitious examples that may lead to an inaccurate

structure and the second disregards the order of the observations. Furthermore, these algorithms work well only when a significant number of examples is provided.

There are two further deficiencies in using Bayesian networks to represent plan libraries. First, the result of a belief update given some evidence is not sensitive to the order in which the evidence is entered in the network. In other words, if actions observed by the agent were performed by the user in a completely different order, the resulting prediction for non-evidenced nodes would be exactly the same. Second, Bayesian networks are not able to handle repetitive actions that give rise to cycles because, by definition, they have an acyclic structure.

Markov models have also been extensively used for modeling activity sequences (Fine et al. 1998; Bui et al. 2002; Blaylock and Allen 2003, 2005; Brand et al. 1997; Soller 2007; Köck and Paramythis 2011). In their simplest form, Markov chains can be derived without any previously known structure. However, the Markov assumption is unrealistic in many domains, including goal recognition. Higher order Markov chains can be used to detect more than very short term memory dependencies in the sequences, at the expense of an exponential growth in the state space as we increase the order of the model. Then, these models are clearly not practical.

Blaylock and Allen (2003, 2005) explored the use of m -gram models to perform statistical goal recognition. Specifically, they built two experiments in the Linux domain using unigram and bigram models, respectively. Then, they defined the problem of goal recognition as a classification task where given an observed sequence of n instantiated actions observed until a given moment, the algorithm searches for the most likely instantiated goal.

Hidden Markov Models (HMM) and their extensions explicitly represent uncertainty in observations. However, when the activities become more complex, or they have long-term temporal dependencies, these models are not adequate. More complex Markov models allow us to provide a better description of the situation being modeled. For example, Abstract Hidden Markov Models (AHMM) (Bui et al. 2002) and Hierarchical Hidden Markov Models (HHMM) (Fine et al. 1998) are capable of handling activities that have hierarchical structure. These models have been used in activity recognition applications such as learning and detecting activities from movement trajectories (Nguyen et al. 2005) and inferring from raw GPS sensor user's daily movements (Liao et al. 2007). These models use approximate inference algorithms for the recognition process. A very rich class of plans can be modeled using these AHMMs due to their ability to structure any number of levels of policies. However, in the same way as with plain HMM, even though parameters estimation from data is possible, the structure of the model must be given beforehand. Furthermore, iterative optimization techniques for learning HMMs fail to encode high order temporal dependencies easily, and often converge in local optima. The training of HMMs is computationally hard and there is no known exact method that can guarantee optimal training within reasonable computing time.

VOM models are an important class of models that extend the well-known Markov chain models. In contrast to the Markov chain models, where each random variable in a sequence with a Markov property depends on a fixed number of random variables, in VOM models this number of conditioning random variables may vary based on the specific observed realization, known as *context*. These models consider that in realistic

settings, there are certain realizations of states (represented by contexts) in which some future states are independent from the past states leading to a great reduction in the number of model parameters.

In contrast to N-order Markov models, which attempt to estimate conditional distributions of the form $Pr(\sigma|s)$, with $s \in \Sigma^N$ and $\sigma \in \Sigma$, VOM algorithms learn such conditional distributions where context lengths $|s|$ vary in response to the available statistics in the training data. Thus, VOM models provide the means for capturing both large and small order Markov dependencies based on the observed data.

Ron et al. (1996) introduced an algorithm for learning VOM models from data. This model is described using a subclass of Probabilistic Finite Automata (PFA), which they called Probabilistic Suffix Automata (PSA). For the construction of the PSA, a construction called Probabilistic Suffix Tree (PST) is used. PST preserves the minimal sub-sequences of variable length that are necessary for precise modeling of the given statistical source.

VOM models have been successfully applied in a variety of domains such as document compression (Begleiter et al. 2004; Nunez-Yanez and Chouliaras 2005), protein classification (Bejerano and Yona 2001; Shmilovici and Ben-Gal 2007), spam filtering (Bratko et al. 2006), natural language recognition (Hu et al. 1997), and behavior modeling from video sequences (Galata et al. 2001). However, the use of VOM models for goal recognition as described in this article is novel.

In the next section we describe some basic concepts of VOM theory needed to understand our approach. Next, in Sect. 4, we show how we apply this theory to build models of user behavior needed to perform goal recognition.

3 Background on VOM models

VOM models were originally defined for data compression by Rissanen (1983). In this section we give some basic definitions and describe the family of distributions used in these models.

The concept of a Markov chain can be generalized through the concept of a PFA (Rabin 1963). A PFA is a generalization of the non-deterministic finite automaton, which includes the probability of a given transition between states into the transition function, turning it into a transition matrix or stochastic matrix.

Formally, a PFA M is a 5-tuple $(Q, \Sigma, \tau, \gamma, \pi)$, where Q is a finite set of states, Σ is a finite alphabet. Each element of the alphabet $\sigma \in \Sigma$ is called *symbol*, $\tau : Q \times \Sigma \rightarrow Q$ is the transition function, $\gamma : Q \times \Sigma \rightarrow [0, 1]$ is the next action probability function, where for each $q \in Q$, $\sum_{\sigma \in \Sigma} \gamma(q, \sigma) = 1$, $\pi : Q \rightarrow [0, 1]$ is initial probability distribution over the starting states, where $\sum_{q \in Q} \pi(q) = 1$.

Machine learning theory gives strong indications that efficient learning of general PFA sources is not possible (Kearns et al. 1994). For this reason, the algorithms for learning such automata are based on the construction of a PST that has been demonstrated to be equivalent to the corresponding PFA and that has roughly the same size (Ron et al. 1996).

3.1 PST

A PST over an alphabet Σ is a non empty tree, whose nodes vary in degree between zero and the size of the alphabet $|\Sigma|$. Each edge in the tree is labeled by a single symbol of the alphabet, such that no symbol is represented by more than one edge branching out of any single node (for this reason, the degree of each node is bounded by the size of the alphabet). The root node is labeled by the empty sequence which we denote by e . The rest of the nodes in the tree are labeled by a sequence of symbols, which is the sequence generated by walking up the tree from that node to the root. Each node labeled by a symbol sequence s is assigned a probability distribution vector over the alphabet, $\gamma_s : \Sigma \rightarrow [0, 1]$, which is called the “next symbol” probability function related to s . It corresponds to the probabilities that the tree assigns to each symbol, given that the longest sub-sequence of symbols that have been observed matches that particular node’s label.

The empirical probabilities are computed as follows. Let $e^1, e^2, \dots, e^m$ be the set of m training examples over the alphabet Σ . The length of the i -th training example is given by l_i , that is $e^i = e_1^i, e_2^i, \dots, e_{l_i}^i \forall e_j^i \in \Sigma$. The empirical probability of a sequence s is presented in Eq. (1)

$$\tilde{P}(s) = \frac{\sum_{i,j} \chi_s^{i,j}}{\sum_{i \text{ s.t. } l_i \geq |s|} (l_i - (|s| - 1))} \quad (1)$$

$$\text{where } \chi_s^{i,j} = \begin{cases} 1 & \text{if } s_1, s_2, \dots, s_{|s|} = e_j^i, e_{j+1}^i, \dots, e_{j+(|s|-1)}^i \\ 0 & \text{otherwise} \end{cases}$$

The numerator is the number of times the sequence s was observed in the sample and the denominator is the maximal number of possible overlapping occurrences a pattern of the same length could have.

The conditional empirical probability of observing an action σ right after a given sequence s is given in Eq. (2)

$$\tilde{P}(\sigma|s) = \frac{\tilde{P}(s \cdot \sigma)}{\tilde{P}(s)} \quad (2)$$

Given a sequence r of observed symbols, its prediction by a certain PST is made symbol by symbol, where the probability of each symbol is calculated by scanning the tree in search of the longest suffix that appears in the tree and ends just before that symbol. The conditional probability of this symbol given this suffix is given by the probability distribution associated with the corresponding node in the PST. Formally, given a sequence $r = r_1, r_2, \dots, r_t$ in Σ^N the probability assigned by a PST to the action sequence r can be computed using Eq. (3)

$$P(r) = \prod_{i=1}^N \gamma(s_{i-1}, r_i) \quad (3)$$

where $\gamma(s_{i-1}, r_i)$ is the probability value assigned by the node s_{i-1} to the symbol r_i , being $s_0 = e$ (the empty sequence) and for $1 \leq j \leq N - 1$, s_j being the sequence labeling the deepest node reached by taking the walk corresponding to $r_i, r_{i-1}, \dots, r_1$ starting from the root of T .

It is easy to see that computing the probability value assigned by a PST to an observed sequence takes a linear time both in the length of the sequence (t) and in the depth of the tree (L), that is $\Theta(L \cdot t)$. For each symbol in the sequence, a run from the root node to the deepest node matching the previous context has to be followed to find the probability distribution that must be used to predict the following symbol. A solution usually adopted to make this computation $\Theta(t)$ (i.e., linear to the length of the observed sequence) is to build a PFA that assigns the same probability distribution assigned by the PST in such a way that, for each observed action, we only have to make a state transition in the automata to obtain the state containing the probability distribution necessary for the prediction. The PFA that can be built from a PST is known as PSA.

3.2 PSA

Formally, a PSA is a subclass of PFA so it can be described as a 5-tuple $(Q, \Sigma, \tau, \gamma, \pi)$, as explained above. Then, a PFA is a PSA if the following property holds: for every two states q_1 and $q_2 \in Q$ and for every action $\sigma \in \Sigma$, if $\tau(q_1, \sigma) = q_2$ and q_1 is labeled by a sequence s_1 , then q_2 is labeled by a sequence s_2 that is a suffix of $s_1 \cdot \sigma$.

Remember that the probability function in each node of the PST is defined for all actions in the alphabet: $\gamma(q, \sigma)$ exists $\forall q \in Q$ and $\forall \sigma \in \Sigma$. If an action σ_i was not observed in the training examples for a given context q_j , a probability value equivalent to γ_{min} is assigned by the next action probability function $\gamma(q_i, \sigma_j) = \gamma_{min}$. This is done to prevent the cumulative probability of a sequence from becoming 0 if the user performs an action in a context not present in the training examples. The preceding property means that the PSA resulting from a PST will be strongly connected so that the transition function will be defined for all pairs of states.

A PSA whose nodes are labeled by sequences of length at most L is known as an L-PSA. Remember that L is the maximum history memory used to build a PST. These automata can be described as Markov Chains of order L whose states are the actions in the alphabet Σ and the next state transition probability depends on the last L states (actions) traversed. This is possible because every L-PSA can be extended to an equivalent L-PSA whose states are labeled by all sequences in Σ^L , although this extension can result in a possible much larger number of states. However, since the states in the L-PSA might be labeled by only a small subset of $\Sigma^{\leq L}$ (due to the two-fold pruning used in the construction of the PST), and since generally many of the suffixes are expected to be much shorter than L , an L-PSA can be viewed as a Markov chain with variable order, or variable memory.

In the next section, we describe how this theory can be used to model user behavior from the observation of the actions a user performs in a software application, and we present an algorithm for building PST models incrementally. We also describe our

approach to perform goal recognition using these models combined with an EMA to improve convergence in the predicted user goal.

4 Modeling and recognition of user behavior with VOM models

While interacting with a software application, users perform sequences of actions that enable them to achieve different goals (i.e., different activities or tasks they need to fulfill). These sequences of actions constitute a very important source of data for automatic user modeling. Since users tend to exhibit recurrent patterns, each pattern corresponding to a particular goal execution, machine learning techniques can be applied to learn a predictive model of the user behavior. That is, we can use a user's traces examples to learn patterns of behavior related to a given goal that the user usually has in the application, and then recognize other new traces as belonging to one of these models of behavior.

For example, given that the user previously gave the agent examples about the sequence of commands he/she performs to achieve a set of goals in the Linux domain, if we observe the user typing `<cd documents>` and `<ls>` in the Linux command line, the agent could infer that the user wants to find a file or subdirectory within the directory named "documents". However, as the user continues executing commands, the agent should update its belief in what the user's goal is. If the user next executes `<cd drafts>`, `<cd thesis>` and `<mv manuscript.tex ../revise>` the agent can infer that the actual user goal was to move the file named "manuscript.tex" to the directory named "revise". This example highlights how our goal recognizer works. After each observed action, the agent should be able to predict (with a certain degree of probability) what the current user goal is, given the observations so far.

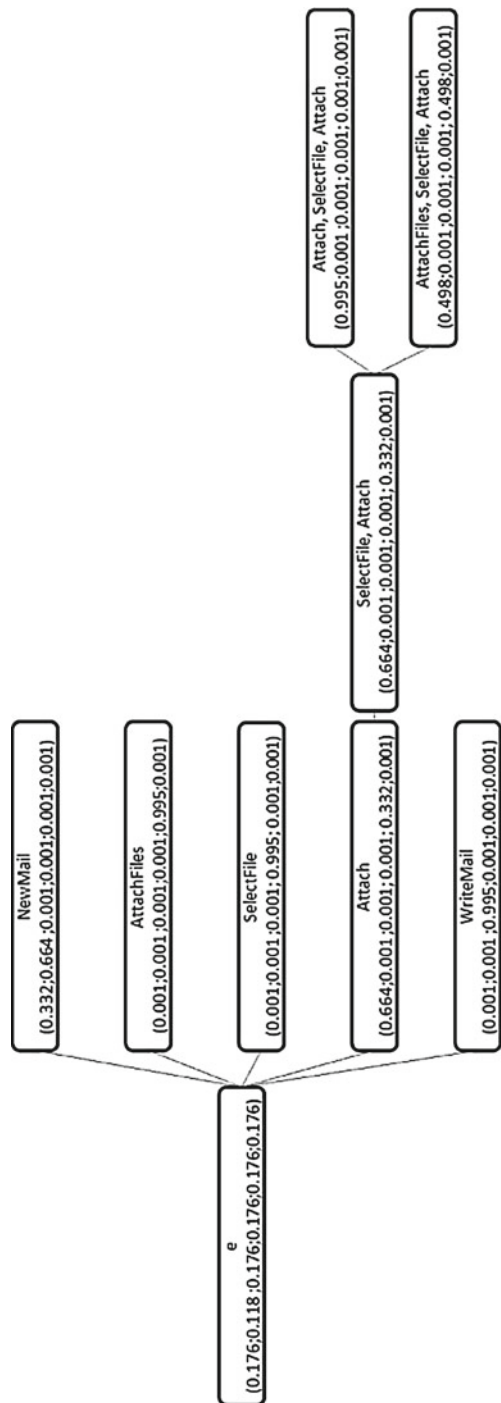
In this section, we explain how we represent sequences of user actions using the VOM theory presented in Sect. 3 and how this representation can be learned from training examples provided by the user. Following this approach, we only need a set of examples of how to achieve a given goal and no extra information about the domain being modeled.

To obtain a probabilistic framework, we consider a sequence of actions performed by the user as a series of random variables $s = s_1, s_2, \dots, s_l$. Each s_j will have a value of one of the possible actions the user can perform in the domain being modeled. Following Markov models terminology, the set of all possible actions the user can perform is called *alphabet* and it is denoted by Σ .

An example of a PST model in the email domain is given in Fig. 1. The model assigns a context for each action in the sequence, and defines the transition probability of each action given this context. Each node has at most $|\Sigma|$ children. Note that the PST is not necessarily balanced or complete.

The tree branches from the root to the leaves (from left to right) represent reversed contexts. In this way, an extension of a branch by adding a node represents the extension of a context by an earlier observed action. For example, the first level node for context `<Attach>` indicates a first order Markov model which is used instead of any longer context for any action, except for *SelectFile*, which along with *Attach* creates a context that better predicts the following action. In this node, we can observe that

Fig. 1 An example of a PST in the email domain. The vector below each node's label corresponds to the next action probabilities for actions WriteMail, AttachFiles, Send, Attach, SelectFile, NewMail, respectively. As can be seen, the alphabet size is 6 in this example



there is a probability value of 0.664 that the user will perform *WriteMail* and 0.332 of observing *SelectFile* just after the observation of the execution of *Attach*. The nodes at level 3 represent a 3rd-order Markov model that consists of the contexts $\langle \text{Attach}, \text{SelectFile}, \text{Attach} \rangle$ and $\langle \text{AttachFiles}, \text{SelectFile}, \text{Attach} \rangle$.

By using the examples provided by the user, the agent will build a PST model for each goal the user can pursue in the domain. When a new example for an existing model is provided, it may correspond to an alternative way of reaching the same goal, so the corresponding model is updated to reflect this fact. This procedure follows a Programming By Example (PBE) approach (Lieberman 2001) in the sense that the user will teach the agent what sequence or sequences of actions he/she usually performs to achieve a given goal. However, unlike the classic PBE approach, our aim is not to create a program that allows the agent to perform repetitive tasks on behalf of the user, but to detect the user's goal that leads him/her to behave in a certain way. Learning the user's patterns of behavior from examples has the main advantage that we do not need any information about the domain being modeled. The agent will be able to learn regularities in the user's behavior just by analyzing the examples provided.

In the next section, we will describe how PST models can be incrementally built from a sequence of actions performed by the user.

4.1 Incremental learning of a PST

Current algorithms for learning PSTs (Ron et al. 1996; Bejerano and Yona 2001) assume that the complete set of training examples is provided *before* the algorithm is run. However, this is usually not the case in the domain of interface agents since the user can provide one example at one moment and additional examples later to refine the model with alternative ways for achieving a goal.

A trivial solution to this problem is to store the training examples and re-run the learning algorithm each time a new example (or a new set of examples) is provided. This solution, although simple, requires the complete tree to be discarded and built again from scratch re-examining all the previous examples, since new sequences of actions might become relevant contexts for predictions and existing sequences in the tree might become irrelevant given the new examples. To avoid discarding the computations made from previous examples, an incremental learning of the PST is desired.

Intuitively, the online algorithm we propose for learning PST has three high level steps that are run for each new example provided:

1. For each symbol σ in the given example, add to the tree a node for every subsequence previous to σ (that is, every possible *context* for the observed action) with length varying from 1 to the minimum between the length of the example and the maximum memory allowed (L). Keep track of the number of times each action was observed in each context.
2. Prune the resulting tree favoring the shortest context that makes “good” predictions.
3. Smooth the resulting prediction probabilities in each node of the tree so that no action is predicted with zero probability.

This approach is similar to the algorithm proposed by Ron et al. (1996), but for the online counting of the observations that allows an incremental learning of the tree. In the rest of this section, we will detail how we adapt the definitions for computing the empirical probability proposed by Ron et al. (1996), and present our algorithm.

The first step for an incremental learning of the PST consists then in keeping track of the number of times each symbol σ is observed in each context s . We denote this count by $\eta_s(\sigma)$. We can then redefine Eq. (1) to compute the empirical probability of a sequence s using $\eta_s(\sigma)$ as shown in Eq. (4),

$$\tilde{P}(s) = \frac{\eta_{\text{prefix}(s)}(\text{last}(s))}{l - (|s| - 1)} \quad (4)$$

where $\text{prefix}(s = \langle s_1, s_2, \dots, s_{n-1}, s_n \rangle) = \langle s_1, s_2, \dots, s_{n-1} \rangle$ corresponds to the prefix of the sequence s , $\text{last}(s = \langle s_1, s_2, \dots, s_{n-1}, s_n \rangle) = s_n$ is the last observed symbol in the sequence and l is the length of the training example.

The conditional probability of an action σ given a context s can be computed as the number of times σ was observed after the context s divided by the sum of the counts of all actions observed in that context, as shown in Eq. (5). Notice that all the information required to compute this value is present in the node labeled by s ,

$$\tilde{P}(\sigma|s) = \frac{\eta_s(\sigma)}{\sum_{\sigma' \in \Sigma} \eta_s(\sigma')} \quad (5)$$

We now present the algorithm we propose for building or updating a PST $\mathcal{T}$ given a new training sequence of actions representing the user behavior for a given goal. Algorithm 1 operates for each training example $s = \langle \sigma_1, \dots, \sigma_n \rangle$ and creates or updates an existing PST $\mathcal{T}$ to model the training sequence. Lines 2 to 5 create the tree root node in case that $\mathcal{T}$ is empty, that is, s is the first observed example for the corresponding goal to be modeled. Then, the algorithm reads one action $\sigma \in s$ at each step (line 6) and searches for all the sub-sequences ss previous to σ in s from length 0 (which corresponds to $ss = \langle \sigma \rangle$ without considering any context) to the minimum between the length of the example sequence $|s| = n$ and the maximum memory L , $ss = \langle \sigma_{\min(|s|, L)}, \dots, \sigma \rangle$ (line 7). If a given subsequence does not exist in the tree, it is added with a zero vector of counts (lines 9–12).

Next, we increment the number of times that σ was observed in the corresponding previous context ($\text{prefix}(ss)$, line 14). Finally, we prune $\mathcal{T}$ (line 18) and compute the smoothed next action probability function (line 19)

Algorithm 2 shows the procedure carried out to prune a given PST $\mathcal{T}$. The aim of pruning the tree is to eliminate those nodes with similar prediction capabilities to other nodes corresponding to shorter contexts (controlled by parameter r), to eliminate nodes corresponding to rarely observed sub-sequences (controlled by parameter $P_{\min}$) and to eliminate nodes that do not predict any action with a significant empirical probability value (controlled by parameters α and $\gamma_{\min}$).

The algorithm traverses the leaves of the tree and checks whether they are relevant to be maintained as part of the PST according to the pruning rules expressed above. If all the children nodes of a node are pruned, the procedure extends to this node

Algorithm 1 Algorithm for learning (or updating) a PST from a sequence of actions

Input: a (possibly empty) PST $\mathcal{T}$, a sequence s over a set of possible actions Σ and a set of parameters P_{min}, γ_{min}, r and L

Output: $\mathcal{T}$ is updated to model S

```

1: procedure UPDATEPST( $\mathcal{T}, s, P_{min}, \gamma_{min}, r, L$ )

2:   if  $\mathcal{T}$  is empty then
3:      $\mathcal{T} \leftarrow$  a single new node labeled with  $e$  (the empty sequence)
4:      $\bar{\eta}_e \equiv \bar{0}$ 
5:   end if

6:   for each action  $\sigma$  in the sequence  $s$  do
7:     for  $j = 0$  to  $\min(|s|, L)$  do
8:        $ss \leftarrow$  subsequence with the  $j$  actions in  $s$  previous to  $\sigma$ 
9:       if there is no node labeled with  $ss$  in  $\mathcal{T}$  then
10:         $n \leftarrow$  a new node labeled with  $ss$ 
11:         $\bar{\eta}_n \equiv \bar{0}$ 
12:        add  $n$  to  $\mathcal{T}$ 
13:      end if
14:      increment  $\eta_{prefix(ss)}(\sigma)$ 
15:    end for
16:  end for
17:  PRUNE( $\mathcal{T}, P_{min}, \gamma_{min}, r$ )
18:  SMOOTH( $\mathcal{T}, \gamma_{min}$ )
19: end procedure

```

(because it turned into a new leaf) and continues until there are no leaves that need to be pruned.

Finally, Algorithm 3 shows the procedure for computing and smoothing the next symbol probability function for each node in $\mathcal{T}$, in the same way it is done in the standard batch algorithm presented in Ron et al. (1996).

This step in the learning algorithm is necessary to assure that no action is predicted with zero probability, regardless of the actions observed before it. The value given by parameter γ_{min} defines the minimum probability for an action observed in any context and the probabilities for other actions in that context should be adjusted accordingly. This is done by decreasing the empirical probabilities such that a total of $|\Sigma|\gamma_{min}$ is collected to be later shared by all the actions in each context. Observe that we must assure that $\gamma_{min} < \frac{1}{|\Sigma|}$ to prevent negative values.

To illustrate the procedure for building a PST from a sequence of actions, consider the following example depicted in Fig. 2. The vector under each node's label corresponds to the observations count for actions *NewMail*, *AttachFiles*, *SelectFile*, *Attach*, *WriteMail*, *Send*, in the given order. Given the training sequence $\langle \text{NewMail}, \text{WriteMail}, \text{Send} \rangle$ the algorithm starts with the node corresponding to the empty sequence. Next, the first action, *NewMail*, is considered. Since this is the first observed action, only one possible context has to be considered: $ss = \langle \text{NewMail} \rangle$. A node for this context is created and the count for *NewMail* is incremented in the node corresponding to the prefix of ss , which is the node corresponding to the empty sequence. For the next observed action, *WriteMail*, two contexts $ss = \langle \text{WriteMail} \rangle$ and $ss = \langle \text{NewMail}, \text{WriteMail} \rangle$ are considered. None of these contexts exists in the tree, so a node is created

Algorithm 2 Algorithm for pruning a PST

Input: a PST $\mathcal{T}$ and a set of parameters P_{min} , γ_{min} and r

Output: a pruned version of $\mathcal{T}$

```

1: procedure PRUNE( $\mathcal{T}$ ,  $P_{min}$ ,  $\gamma_{min}$ ,  $r$ )
2:   for each node  $s$  from the leaves to the root of  $\mathcal{T}$  do
3:     if  $s$  has none unpruned children and not RELEVANT( $s$ ) then
4:       prune  $s$ 
5:     end if
6:   end for
7: end procedure

8: function RELEVANT(subsequence)
9:   if  $\tilde{P}(\text{subsequence}) \geq P_{min}$  and there exists an action  $\sigma$  such that
10:    (I)  $\tilde{P}(\sigma | \text{subsequence}) \geq \gamma_{min}$  and
11:    (II)  $\frac{\tilde{P}(\sigma | \text{subsequence})}{\tilde{P}(\sigma | \text{suffix}(\text{subsequence}))} \begin{cases} \geq r \\ \text{or} \\ \leq \frac{1}{r} \end{cases}$  then
12:      return true
13:   else
14:     return false
15:   end if
16: end function

```

Algorithm 3 Algorithm for smoothing probabilities in a PST

Input: a PST $\mathcal{T}$ and a set of parameters P_{min} , γ_{min} , r and L

Output: a pruned version of $\mathcal{T}$

```

1: procedure SMOOTH( $\mathcal{T}$ ,  $\gamma_{min}$ )
2:   for each  $s$  labeling a node in  $\mathcal{T}$  and each action  $\sigma \in \Sigma$  do
3:      $\gamma_s(\sigma) = (1 - |\Sigma|\gamma_{min}) \frac{\eta_s(\sigma)}{\sum_{\sigma' \in \Sigma} \eta_s(\sigma')} + \gamma_{min}$ 
4:   end for
5: end procedure

```

for each of them. For the 1-order context the count in the node corresponding to the empty sequence is incremented. For the 2-order context, the count in the node corresponding to its prefix, $\langle \text{NewMail} \rangle$, is incremented. A similar procedure is followed for the third observed action, where the algorithm considers a memory of 1, 2 and 3 previous observations. In Fig. 2 we can observe the resulting tree.

Once the tree skeleton is constructed, the algorithm traverses the tree to eliminate those nodes with prediction probabilities that are similar to their parent nodes. Suppose that the pruning procedure starts with node $\langle \text{NewMail} \rangle$, with $P_{min} = 0.01$, $\gamma_{min} = 0.001$ and $r = 1.05$. All three relevance rules succeed and the node is kept in the tree:

$$\tilde{P}(\langle \text{NewMail} \rangle) = \frac{\eta_{\langle e \rangle}(\text{NewMail})}{|s| - (|\langle \text{NewMail} \rangle| - 1)} = \frac{1}{3} \geq P_{min}$$

$$\tilde{P}(\text{WriteMail} | \langle \text{NewMail} \rangle) = \frac{\eta_{\langle \text{NewMail} \rangle}(\text{WriteMail})}{\sum_{\sigma' \in \Sigma} \eta_{\langle \text{NewMail} \rangle}(\sigma')} = 1 \geq \gamma_{min}$$

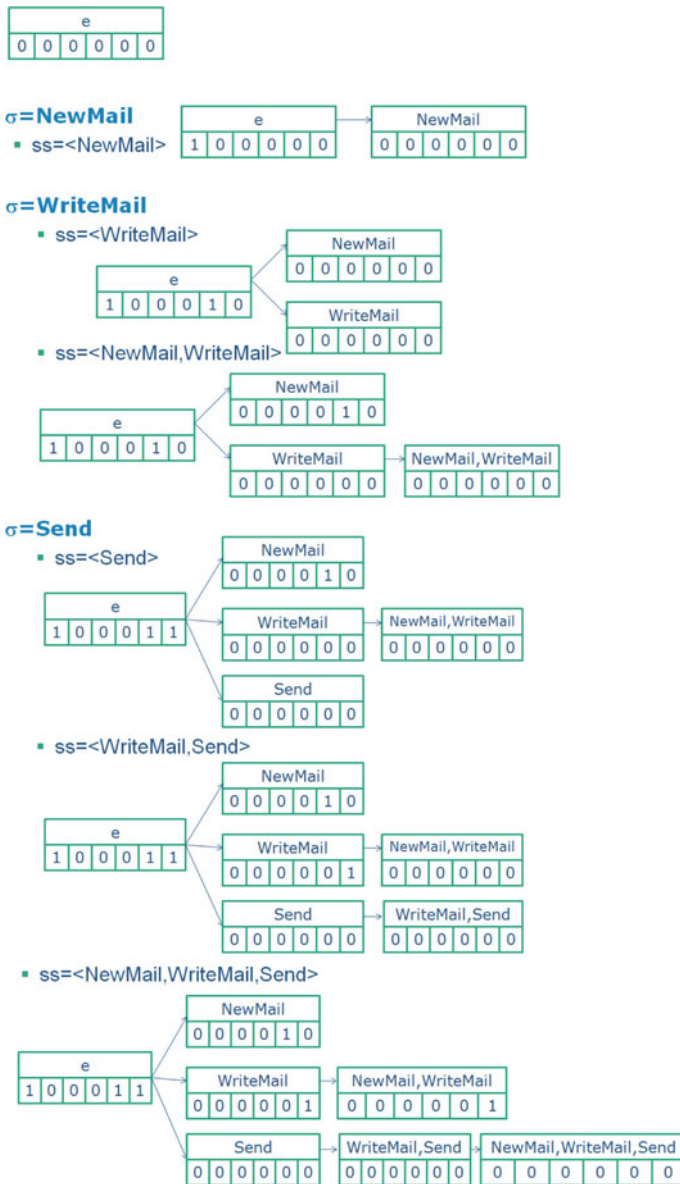


Fig. 2 PST construction example from the sequence $\langle \text{NewMail}, \text{WriteMail}, \text{Send} \rangle$

$$\frac{\tilde{P}(\text{WriteMail} | \langle \text{NewMail} \rangle)}{\tilde{P}(\text{WriteMail} | \langle e \rangle)} = \frac{1}{0.33} \geq r$$

The following leaf to consider is the one labeled by $\langle \text{NewMail}, \text{WriteMail} \rangle$. For this node, the last relevance rule fails because there exists at least one action, *Send*,

for which the prediction function in this node is the same as the prediction function in its parent corresponding to a shorter context ($\langle \text{WriteMail} \rangle$)

$$\frac{\tilde{P}(\text{Send}|\langle \text{NewMail}, \text{WriteMail} \rangle)}{\tilde{P}(\text{Send}||\langle \text{WriteMail} \rangle)} = 1 \begin{cases} < r \\ > \frac{1}{r} \end{cases}$$

Consequently, the node labeled by $\langle \text{NewMail}, \text{WriteMail} \rangle$ is pruned. Its parent, labeled by $\langle \text{NewMail} \rangle$ is checked and all three relevance rules succeed.

For the node labeled by $\langle \text{NewMail}, \text{WriteMail}, \text{Send} \rangle$ the second relevance rule fails, since the probability of occurrence of this context is lower than the minimum accepted γ_{min} . This node is pruned and its parent node is considered. For this node, the second relevance rule also fails and is pruned accordingly. Finally, the node labeled by $\langle \text{Send} \rangle$ is also pruned since the empirical probability of this context is also lower than the minimum accepted to be maintained in the PST.

After this process, a PSA equivalent to the PST is constructed in the form presented by Ron et al. (1996) using the information contained in the tree and no extra information. In the following subsection we summarize this procedure.

4.2 Building a PSA from a PST

To build a PSA, we start by using the set of nodes in the PST taken as the states in the PSA. The first problem that arises is that we may not be able to define the transition function for all actions in every state because the destination state might not exist. In other words, there might exist a state s and an action σ such that there is no state s' that is a suffix of $s \cdot \sigma$. The solution proposed in the literature (Ron et al. 1996) is to extend the set of states to contain all the *prefixes* of the nodes in the PST that are not part of the PST.

The transition function is defined as follows: for every state s and action σ , $\tau(s, \sigma)$ is the state labeled by the longest suffix of $s \cdot \sigma$. The next action probability function is defined as follows. For every state s corresponding to a node q in the PST and for every action σ we use the same next action probability function in q , $\gamma_{PSA}(s, \sigma) = \gamma_{PST}(q, \sigma)$. For each new state added to the PSA and for every action σ we let $\gamma_{PSA}(s, \sigma) = \gamma_{PST}(s', \sigma)$, where s' is the longest suffix of s which is a node in the PST. This definition leads the PSA to generate the same probability distribution as the PST, since we are using the deepest ancestor of s in the PST. Finally, the state corresponding to the empty sequence is set to be the unique initial state in the PSA by setting $\pi(e) = 1$. Algorithm 4 summarizes this procedure.

To illustrate the procedure for building a PSA from a PST shown in Algorithm 4 we will build the PSA corresponding to the PST shown in Fig. 1.

We start with an empty set of states Q (step 1) and iterate over the nodes in the PST $\mathcal{T}$. We start with the root node and create the corresponding state in the PSA (step 3) with the same next action probability function as the root node in the PST (step 4). Since the empty sequence has no prefixes, we simply add the new created state to Q . Next, we take, for example *NewMail*, create a new state in the PSA labeled by this sequence of length one and assign the same next action probability function

Algorithm 4 Algorithm for building a PSA from a given PST

Input: A PST $\mathcal{T}$

Output: a PSA $\mathcal{M}$ generating a probability distribution equivalent to $\mathcal{T}$

```

1:  $Q \leftarrow \emptyset$ 
2: for each sequence  $q_s$  labeling a node  $q$  in  $\mathcal{T}$  do
3:    $s \leftarrow$  new node labeled with  $q_s$ 
4:   define  $\gamma_{\mathcal{M}}(s, \sigma) = \gamma_{\mathcal{T}}(q, \sigma), \forall \sigma \in \Sigma$ 
5:    $Q \leftarrow Q \cup \{s\}$ 

6:   if there is no node labeled with  $prefix(q_s)$  in  $\mathcal{T}$  then
7:      $s_{prefix} \leftarrow$  new node labeled with  $prefix(q_s)$ 
8:      $s' \leftarrow$  node labeled with the longest suffix of  $q_s$  in  $\mathcal{T}$ 
9:     define  $\gamma_{\mathcal{M}}(s_{prefix}, \sigma) = \gamma_{\mathcal{T}}(s', \sigma), \forall \sigma \in \Sigma$ 
10:     $Q \leftarrow Q \cup \{s_{prefix}\}$ 
11:   end if

12: end for

13: for each state  $s \in Q$  do
14:   for each action  $\sigma \in \Sigma$  do
15:     define  $\tau_{\mathcal{M}}(s, \sigma) =$  longest suffix of  $s \cdot \sigma$ 
16:   end for
17: end for
```

as the node labeled by *NewMail* in the PST. A sequence of length one has a unique prefix that is the empty sequence which is a node in $\mathcal{T}$, so nothing else is done for this sequence. In a similar way, the states corresponding to all sequences of length one in the PST are added.

Next, we take the only sequence of length two in $\mathcal{T}$: $\langle \text{SelectFile}, \text{Attach} \rangle$ and add a corresponding state in $\mathcal{M}$ with the same next action probability function as the corresponding node in $\mathcal{T}$. The only prefix of this sequence is $\langle \text{SelectFile} \rangle$, which has a corresponding node in $\mathcal{T}$, so we move on.

We then take the node in $\mathcal{T}$ labeled by $\langle \text{Attach}, \text{SelectFile}, \text{Attach} \rangle$, create the corresponding state and next action probability function and see that its prefix $\langle \text{Attach}, \text{SelectFile} \rangle$ does not have a corresponding node in the PST. A new node labeled by this prefix is created (step 7) and we assign to this node the next action probability function of the node labeled by the longest suffix of $\langle \text{Attach}, \text{SelectFile} \rangle$ in $\mathcal{T}$ (steps 8 and 9), which turns to be $\langle \text{SelectFile} \rangle$.

Something similar happens when we add the last node in $\mathcal{T}$ corresponding to the sequence $\langle \text{AttachFiles}, \text{SelectFile}, \text{Attach} \rangle$: besides adding a new state corresponding to this sequence, we create a new state labeled with the sequence $\langle \text{AttachFiles}, \text{SelectFile} \rangle$, and assign the same next action probability function as the node in the PST labeled by $\langle \text{SelectFile} \rangle$.

Finally, the transition function is defined for each state $q \in Q$ and for each action $\sigma \in \Sigma$ (steps 13–17) as the longest suffix of the sequence $s \cdot \sigma$, where s is the sequence labeling the node q . For example, the next state for the node labeled with $\langle \text{AttachFiles}, \text{SelectFile} \rangle$ for the action *Attach* will be the state labeled by $\langle \text{AttachFiles}, \text{SelectFile}, \text{Attach} \rangle$ and the next state for the node labeled by $\langle \text{AttachFiles}, \text{SelectFile}, \text{Attach} \rangle$ for the action *WriteMail* will be the state labeled by $\langle \text{WriteMail} \rangle$ since there is no longer suffix of $\langle \text{AttachFiles}, \text{SelectFile}, \text{Attach}, \text{WriteMail} \rangle$ in $\mathcal{M}$.

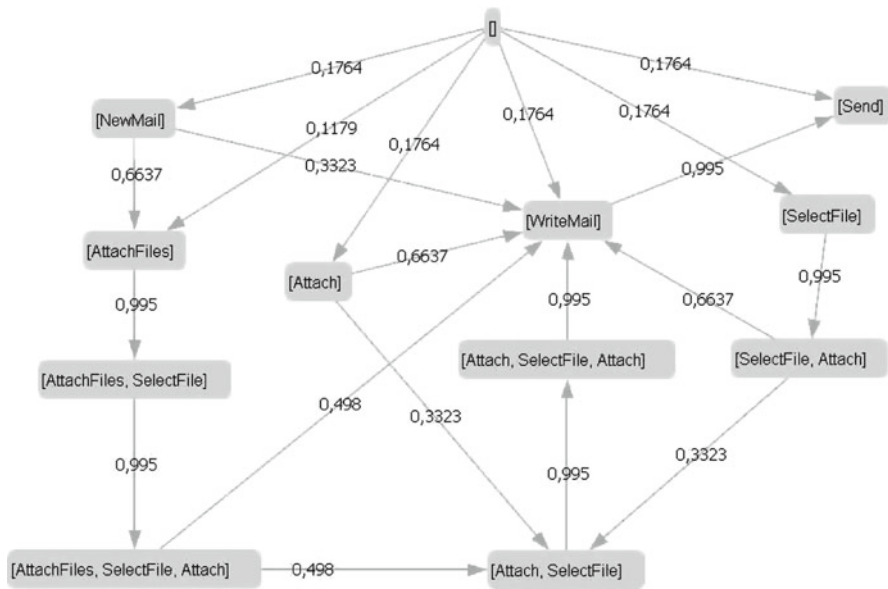


Fig. 3 PSA corresponding to PST in Fig. 1

Figure 3 shows the PSA resulting from applying Algorithm 4 to the PST in Fig. 1. Although the PSA is fully connected, arcs with the minimum probability γ_{min} are not shown in the graph for clarity.

4.3 Matching observations to behavior models

In order to detect the user's goal, during the normal interaction of the user with the software application, the agent will try to match the observed sequence of actions to each PSA model of user's behavior. To this end, the agent will have a PSA model for each goal for which it was trained by means of examples provided by the user. By having a separate model for each goal, the agent will be able to track several goals that are being pursued simultaneously by the user. In the remainder of this section, we will explain how to calculate the degree of similarity of the user's current behavior with models available to the agent.

Matching an observed sequence of actions $r = r_1, r_2, \dots, r_t$ to a PSA model can be done starting from the root node and making the transitions corresponding to the action observed at each time-step. The classical approach for classification of sequences using PSAs computes the probability assigned by a PSA to a sequence r using Eq. (6)

$$P_{PSA}(r) = \prod_{i=1}^t \gamma(s_{i-1}, r_i) \quad (6)$$

where $s_0 = e$, the root state labeled with the empty sequence, and for $1 \leq j \leq t$, $s_j = \tau(s_{j-1}, r_i)$.

This classical approach assumes that the length of each sequence is fixed and the class matching the observations is given only after observing the complete sequence. However, as the user continues executing actions, the total cumulative probability value assigned by each PSA will become smaller after each observation since we are multiplying values in the range $(0, 1]$. The agent will be faced with a continuous stream of observations with uncertain limits among different user goals, and should be able to recognize changes in the user's current goal. Moreover, the recognition process should not be affected by the execution of noisy actions.

In fact, goal recognition is not a classical classification problem since we do not predict a "class" (goal) after observing a complete sequence of performed actions. An interface agent should be able to predict the most probable user goal after each observed action, and the boundaries between sequences of actions corresponding to different goals are frequently fuzzy.

Inasmuch as transitions between states in the PSA modeling the current user goal will have higher probability values than transitions between states in the rest of the PSAs modeling other goals, we choose to use an *EMA* on the prediction probability $\gamma(s, \sigma)$ at each step in each PSA as the predicted value for each corresponding user goal. EMA is a smoothing technique often used for reducing the effect due to random variation on the collection of data taken over time. This technique, when properly applied, reveals more clearly the underlying trend in the observations. EMA is an *amnesic* scheme that allows us to give more importance to the more recent observation and gradually forget about actions executed in the past.

The smoothed prediction is noted with EMA_i . EMA_1 is set to $\gamma(s_0, \sigma_1)$, where $\gamma(s_0, \sigma_1)$ stands for the original prediction in the root state s_0 for the first observed action σ_1 . The subscripts refer to the time periods, $1, 2, \dots, n$. For the second period, $EMA_2 = \lambda\gamma(s_1, \sigma_2) + (1 - \lambda)EMA_1$, and so on.

In general, for any time period $t \geq 2$, the smoothed value EMA_t is found by computing Eq. (7)

$$EMA_t = \lambda\gamma(s_{t-1}, \sigma_t) + (1 - \lambda)EMA_{t-1} \quad (7)$$

Equation (7) is the basic equation of exponential smoothing, or EMA, and the constant or parameter λ is called the smoothing constant. It is easy to observe that the contribution to the smoothed value EMA_t decreases at each consecutive time period. The speed at which older observations are forgotten (smoothed) is a function of the value of λ . When λ is close to 1, forgetting is quick and when λ is close to 0, forgetting is slow. In other words, a value of $\lambda = 1.0$ implies that only the most recent measurement influences the EMA, that is, $\gamma(s_{t-1}, \sigma_t)$. Thus, a large value of λ gives more weight to recent observations than to older probabilities; a small value of λ gives more weight to older observations.

The smoothing constant λ may be expressed as a percentage, so a smoothing factor of 10% is equivalent to $\lambda = 0.1$. Alternatively, λ may be expressed in terms of N time periods, where $\lambda = \frac{2}{N+1}$. For example, $N = 19$ is equivalent to $\lambda = 0.1$.

The value of λ is usually set between 0.2 and 0.3 (Hunter 1986) although this choice is somewhat arbitrary and λ should be determined empirically for each application domain.

To sum up, the goal recognition process works as follows: as the user interacts with the software application, the agent will keep making the transitions corresponding to each observed action in each PSA and computing the EMA of the transition probability given by each PSA. At each time-step, the agent will own a probabilistically ranked set of PSAs which corresponds to the most probable goals the user may have at each moment. It is up to the logic of the interface agent to determine how to make use of this information to assist the user.

In the following section, we present the experiments we conducted using data collected from real users in the Linux domain. We analyze the accuracy and convergence of our approach and compare the results obtained with Fixed Order Markov models. We also empirically estimate the value of the smoothing constant in the EMA calculation that leads us to obtain better recognition performance.

5 Experimental evaluation

Although many corpora are available for testing Machine Learning algorithms for other actions such as classification or clustering, corpora for training goal recognizers are hard to obtain. There are a few plan recognizers (Lesh 1998; Bauer 1999; Garland and Lesh 2002; Stumpf et al. 2005; Blaylock and Allen 2005) that use corpora to build the plan library. There are a few others (Davison and Hirsh 1998; Gorniak and Poole 2000) for which the sequences of actions collected are not labeled with the user goal and which are not used for goal recognition but for next action/command prediction. Moreover, plan recognizers that do use plan corpora to learn the plan library use their own data which makes their comparison difficult.

In this article, we extend results presented in (Armentano and Amandi 2009) with results from a concrete domain and with data collected from real users. For our experiments we selected the Linux Plan Corpus (Blaylock 2005). This plan corpus is modeled after Lesh's Unix Plan Corpus (Lesh 1998) and was collected from 56 human Linux users at the University of Rochester's Department of Computer Science. Users were volunteer students, faculty and staff with different levels of expertise in the use of Linux, categorized from 1 (lowest) to 5 (highest).

Each user was given English descriptions of a set of goals (such as "find a file that ends in.tex" and was instructed to reach each of these goals using any Linux commands. Users were asked not to use pipes or any special commands which would simplify the achievement of the desired goal. Users were given the possibility of performing more than one session for each goal. All sessions, consisting in the sequence of commands performed by a user to achieve a given goal, were automatically recorded. At the end of the session, the user was asked to indicate whether he/she succeeded or not in achieving the goal. The corpus consisted of 457 successful plan sessions with 19 goal schemas.

5.1 Pre-processing

The first step in our experiments was to pre-process the raw user sessions. From the data recorded for each user session, we were only interested in the user goal and in the

sequence of commands he/she performed. We automatically removed commands with typos from each session. Attributes, flags and parameters for the rest of the commands were removed so as to keep the name of the command as action schemas. There was an exception for some commands, such as *ls* and *find* for which some flags changed the command functionality. For these commands we created more than one action schemas as detailed below.

The command *find* searches the directory tree rooted at each given file name by evaluating a given expression. This command was split into four action schemas:

- *find-ctime*, representing the command “*find [path...] -ctime n*”. This attribute instructs to search for files whose status was last changed $n \times 24$ h;
- *find-name*, representing the command “*find [path...] -name pattern*”. This attribute instructs to search for files whose names match the given pattern;
- *find-size*, representing the use of the command “*find [path...] -size s*”. This attribute instructs to search for files using *s* units of storage space;
- *find* grouping all other uses of the command.

The command *ls* lists information about files in the current directory by default. This command was split into two action schemas:

- *ls-R*, representing the command “*ls-R*”. This attribute instructs to list information about files in the base directory and recursively in its subdirectories;
- *ls* grouping all other uses of the command.

The resulting alphabet consisted of 48 command schemas.

Next, we decided to split the Linux corpus onto a by-user basis because different users may have different approaches to achieve the same goal. We are not interested in modeling the Linux domain *per se* but to model the sequences of actions a user has to perform to achieve a given goal. For this reason, sequences performed by different users to achieve the same goal should not be merged into a single model.

None of the 56 users in the dataset successfully completed the 19 goal schemas. For our study we only kept those users who successfully completed more than five goals and who provided more than one example for each goal. The set of 56 users were then reduced to 18 representative users.

The number of commands used by those 18 users to achieve each goal ranged from 1 to 61, with a general average of 6.2 actions. We did not include in the experiments goals achieved using only one command. We also excluded those sequences of actions for which the user explicitly indicated that he/she could not succeed in achieving the given goal.

5.2 Goal recognition performance

We evaluated our approach in terms of *accuracy* and *convergence* (Blaylock and Allen 2003) as follows:

- *Online accuracy*. For a certain goal, online accuracy is defined as the number of time-steps in which the user's current goal was the most probable goal divided by the number of performed actions. In other words, if we assume that the recognizer

makes predictions after each observed action, the online accuracy of an observed sequence $Seq = \sigma_1, \sigma_2, \dots, \sigma_N$ for a given user goal q is the number of correct predictions over the total number of observed actions, as shown in Eq. (8). $q(\sigma)$ is the resulting EMA value assigned by PSA q after observing action σ given the previous context, and $q_{best}(\sigma)$ is the highest resulting EMA value assigned by all PSAs after observing action σ given the previous context

$$accuracy_{online}(q, Seq) = \frac{\sum_{i=1}^N best_q(\sigma_i)}{N} \quad (8)$$

$$\text{where } best_q(\sigma_i) = \begin{cases} 1 & \text{if } q(\sigma_i) = q_{best}(\sigma_i) \\ 0 & \text{otherwise} \end{cases}$$

- *Accuracy*. The standard accuracy of a model q to recognize a given sequence Seq indicates whether the sequence is recognized by q when it has been observed completely.

$$accuracy_{standard}(q, Seq) = \begin{cases} 1 & \text{if } q(\sigma_N) = q_{best}(\sigma_N) \\ 0 & \text{otherwise} \end{cases}$$

- *Convergence*. When applied to a specific goal, this criterion indicates the average number of observations, after which a recognized goal converges to the correct answer, over the average number of observations for those traces that converged. It measures how fast the recognition process of a goal converges to the correct answer. This metric is only applicable when the plan recognizer ends up predicting the actual user goal. If the algorithm predicted correctly the actual user goal from the time-step t to the time-step corresponding to the last performed action, then the convergence is computed as shown in Eq. (9). The time-step t is called *convergence point*

$$convergence(q, seq) = \frac{N - t + 1}{N}, \quad (9)$$

$$\begin{aligned} ¬\ best_q(\sigma_{t-1}), \\ &best_q(\sigma_j), \quad \forall j\ t \leq j \leq N \end{aligned}$$

$$\text{where } best_q(\sigma_i) = \begin{cases} 1 & \text{if } q(\sigma_i) = q_{best}(\sigma_i) \\ 0 & \text{otherwise} \end{cases}$$

We ran our experiments by doing a *leave-one-out cross validation* over the collected examples for each user. We tested different values for the smoothing constant, ranging from 0.1 to 1.0 with intervals of 0.1. Figure 4 shows the resulting average values for all 18 users in the experiment.

We can see that we obtained better convergence and standard accuracy with a value of $\lambda = 0.3$. This is equivalent to a (smoothed) sliding window on the last five observed actions (remember that λ can be expressed as $\lambda = \frac{2}{N+1}$, where N is the number of last actions that concentrate the highest weights in the EMA computation). As expected, accuracy (both online and standard) is higher than convergence in all cases since the

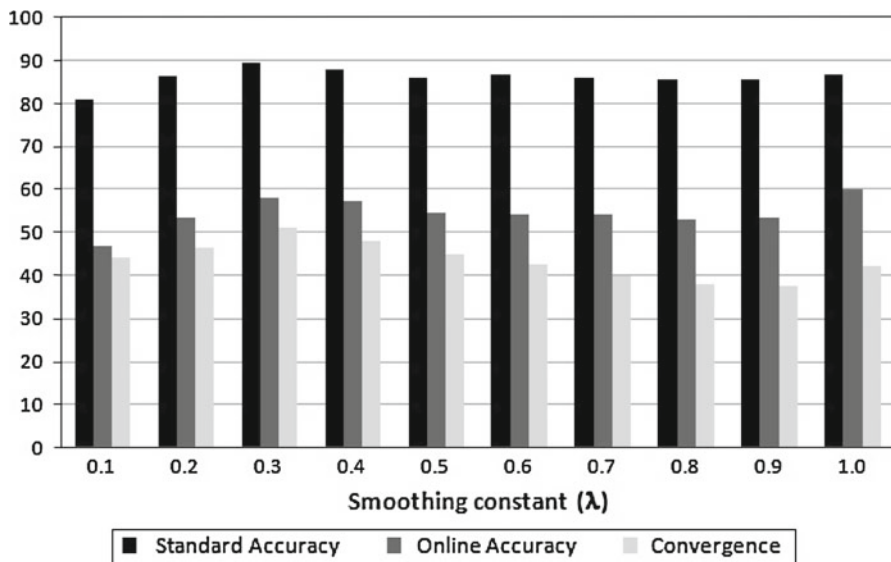


Fig. 4 Accuracy and convergence for different values of the smoothing constant λ

recognizer is often “confused” at the beginning of the observed sequence, making some good and bad predictions until it finally converges. With $\lambda = 0.3$ convergence is reached on average when half the sequence has been observed, confirming our hypothesis that the smoothing constant enables an earlier detection of the user’s current goal. We can observe that around 90% of the sequences finally converge to the user’s actual goal.

On the contrary, better results for online accuracy were obtained without considering the smoothing factor (that is with $\lambda = 1.0$). This reflects the fact that some commands are very frequently used at the beginning of the sequences performed to achieve many of the proposed goals. These commands include *ls*, *cd* and *find*, which are widely used in many situations to establish the context in which the user will start the real completion of his/her goal. Five out of the 18 users in the experiment did not employ these general commands at the beginning of the sequences or employed more specific variations of these commands such as *find-name*, *find-size*, *ls-R*. For those users, a value of the smoothing constant between 0.3 and 0.4 gave better results for both accuracy measures.

In order to compare these results with competing approaches, we performed the same experiments using fixed-order Markov models and HMM. Figure 5 shows a comparison of accuracy and convergence obtained for Markov models of order 1, 2 and 5 (indicated as 1-Markov, 2-Markov and 5-Markov in Fig. 5, respectively), HMM with 2, 3 and 5 hidden states (indicated as HMM-2, HMM-3 and HMM-5 in Fig. 5, respectively), and those obtained with VOM models with EMA smoothing (VOMM+EMA).

We can see that VOM models outperformed fixed order Markov models and HMM for the three metrics considered. Except for standard accuracy, for which 2-Markov models outperformed 1-Markov and 5-Markov, we can observe that as we increment

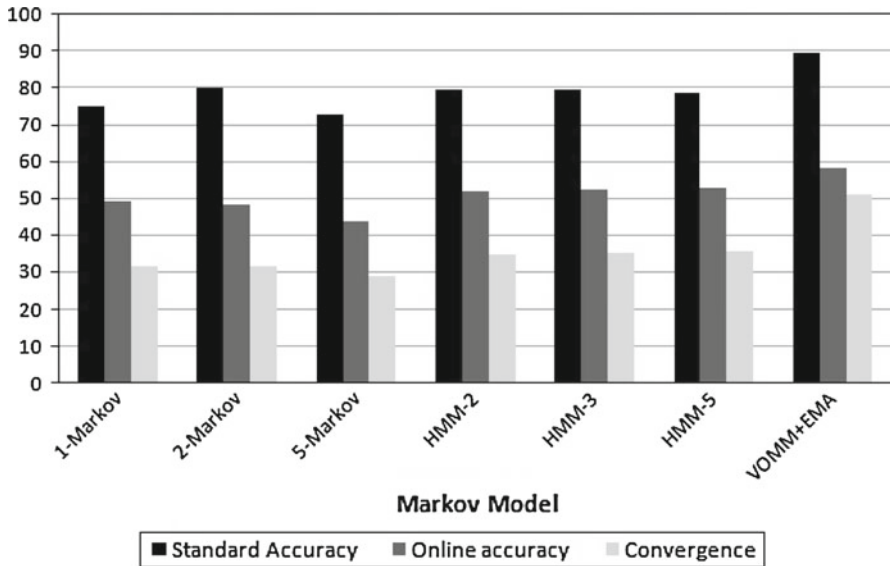


Fig. 5 Comparison of VOM models with fixed-order and hidden Markov models

the order of the models, we get a worse performance for both accuracy and convergence with fixed order Markov models. HMM give a slightly better performance when compared to fixed order Markov models, and tend to keep similar average performance when we vary the number of hidden states. By using VOM models with EMA smoothing we obtained 89.55% accuracy, 58.13% online accuracy and 51.01% convergence, an improvement of 10.92, 5.17 and 15.37% respectively as compared to HMM-5, which resulted to be the best competing model.

We also tested the performance of our approach using *n*-best predictions, that is, considering that the algorithm selects the *n* best ranked models as the predicted user behavior. Figure 6 compares the results obtained for VOM models with EMA smoothing when predicting the top ranked model (1-best) with the results obtained when predicting using the two and three best ranked models.

We can see that 96.18 and 98.47% standard accuracy can be reached if we consider 2-best and 3-best predictions, respectively. Similarly, online accuracy is increased to 73.31% for 2-best predictions and to 83.57% for 3-best predictions. Finally, convergence can reach values of 55.50 and 68% if we predict the two and three best ranked models, respectively.

As an example of the way VOM models capture different context lengths, Fig. 7 shows the VOM model created for the goal *move-files-by-attr-size-lt* for user 04. We can observe that different lengths have been used in different contexts. For instance, we can see that if the user executes command *find-size*, then he/she will probably complete the given goal using command *mv*, but if the user starts with command *ls*, then he/she will probably continue executing *ls* commands until finding the file he/she desires to move.

In Fig. 8, we expand results presented in Fig. 5 to show the performance of the plan recognizer for each user in the experiment.

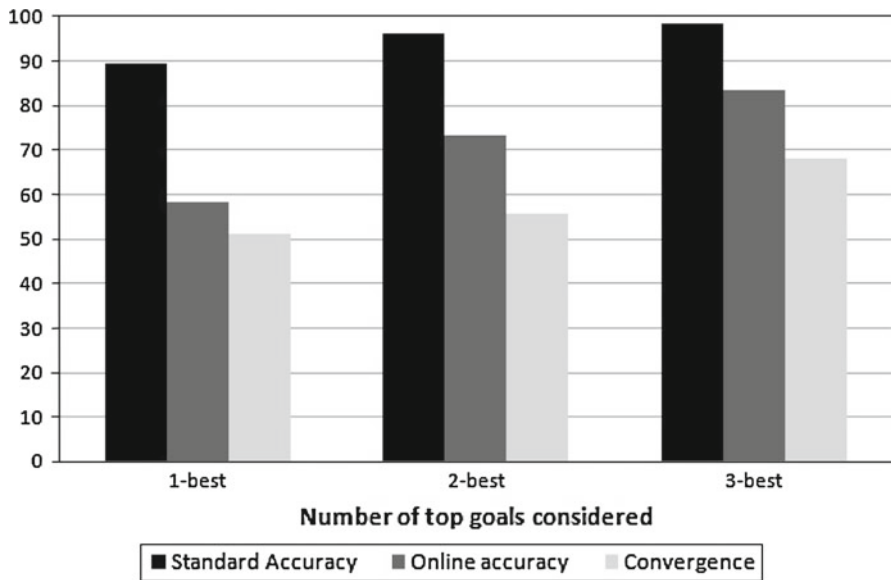


Fig. 6 n-best predictions using VOM models with EMA smoothing

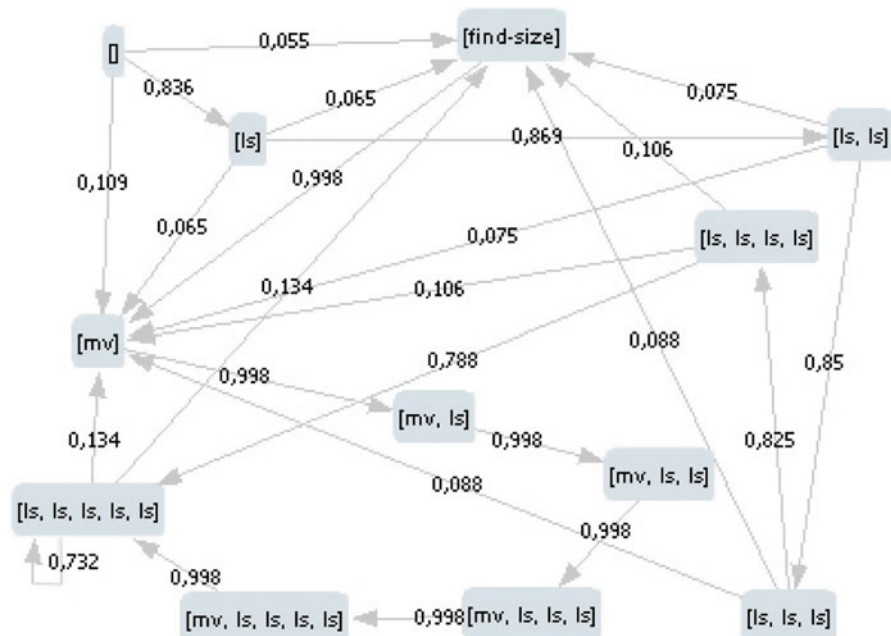


Fig. 7 An example of a VOM model created for goal *move-files-by-attr-size-lt*

We can observe that there is a considerable variation in the results obtained for some users. The standard deviation for convergence is 14.7%, for online accuracy 11.4% and for standard accuracy 15.2%. Those users for whom the recognizer behaved worst

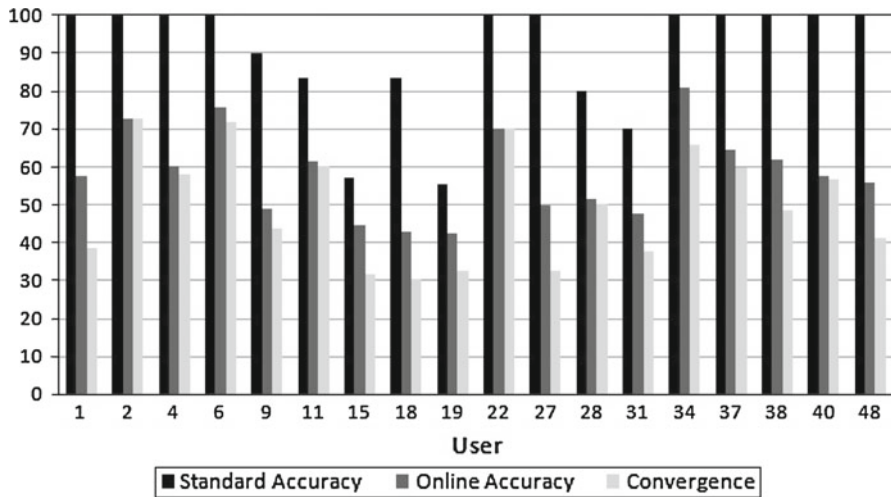


Fig. 8 Average performance for each user

are users 15, 18, 19, 28 and 31. By analyzing these users we found that there were some goals for which the goal recognizer did not converge. These goals have in common that they were trained with very short sequences of two, three and four commands. These sequences are not long enough to take advantage of the variable order part of our models since there is not enough statistical information to learn from. Moreover, when using the sequences that were left out from model training in model testing, these short sequences do not “activate” the model memory to start making good predictions.

A further drawback in the experiment was the number of examples provided by each user to achieve the proposed goals. Many of the 56 users in the experiment only provided one example for each goal. We had to discard those goals since we could not perform cross validation when only one example is available for training. Many of the 18 users we kept for our experiments only provided two examples for each goal. This fact affected the performance of the goal recognizer when the procedure followed by the user to achieve a given goal was very different in each case (sometimes using very different commands in each example provided for the same goal). For these users, the performance decreased drastically since we only had the chance of training a model with one sequence and testing the model with the remaining (very different) sequence.

In general, we can conclude that our plan recognizer works better for longer sequences than for shorter ones. The models that obtained the lowest performance in the experiments correspond to those trained with sequences shorter than five actions. Furthermore, the presence of some models trained with a short sequence and few commands affected the performance of other models trained with the same commands and longer sequences. This is due to the fact that the execution of one of those “shared” commands will have much more weight in the EMA computation in the model trained with a shorter sequence than in the other models because of the way in which the empirical probabilities are computed. However, our approach proved to be effective in making early predictions in cases where, on average, half of the sequence aiming

to achieve a goal had been observed. We believe that, in applications that need longer sequences to achieve goals, our proposed approach will be able to reduce the number of observations needed to converge.

6 Conclusions

In this article we addressed the problem of building a model of user goals from sequences of observed actions performed by the user. We found that VOM models are a good alternative for capturing regularities in the sequence of actions performed by the user to accomplish his/her goals. These models extend the well-known Markov chains by making variable the order of the model. In this way, longer order contexts are used only if they enable better predictions than shorter contexts. We proposed an incremental learning algorithm for building PSTs, which are a data structure needed to efficiently learn VOM models.

Then, we addressed the problem of inferring the user's intention using PSA, which are an alternative representation of PSTs that enable fast predictions linearly in the number of goals. We also proposed the use of an EMA for smoothing the predicted probabilities in order to better capture the underlying tendency in the goal followed by the user.

Finally, we evaluated the proposed model with data collected from real users performing actions in the Linux domain. We conclude that our approach facilitates an early and accurate prediction of the user's goal, when only half of the length of the sequence to achieve a given goal has been observed. We obtained an improvement of 10.92% in accuracy, 5.17% in online accuracy and 15.37% in convergence compared to HMM with 5 hidden states. However, we believe that the performance can be improved even further if the models are trained with longer sequences to take advantage of the variable part of VOM models to represent longer contexts. Goal recognition was difficult to perform in the selected domain because the number of commands performed to achieve the desired goals was low and the training sequences were generally short, not allowing in many cases the activation of the model memory. However, our plan recognizer proved to perform well in this domain.

One limitation of our approach is that the user has to provide examples of how he/she achieves goals in the software application. Many users might not be willing to provide enough examples for the interface agent to build an accurate model of his/her goals. Expressiveness is also reduced since we are not able to model goal abstractions. In many situations, it might be difficult to determine which goal, from a subset of goals consistent with the user actions, is currently being pursued by the user. However, if these goals are similar in essence, it might be useful to predict some abstraction instead of the specific goal being pursued. For example, in our case study, there were three goals related to finding files with different attributes. These goals were modeled separately although they can be abstracted to a general goal "*find a file*" and might even share some subsequences of actions that lead to their completion. Nevertheless, providing goal abstractions requires extra work from the user, which might not be welcomed. On the other hand, an automated abstraction is a difficult task if no domain information is available.

A second drawback of our approach is that it does not handle parametrized goals. Symbols in the alphabet for the VOM model training algorithms are atomic and finite. A simple solution might be to extend the number of symbols to include every combination of observed parameters as in (Madani et al. 2009). However, this approach produces an explosion in the number of symbols and also data sparseness.

Finally, VOM models are sensitive to the number of actions in the training sequences. On the one hand, if the provided training sequences are too short, we cannot take advantage of the variable order part of our models, since there is not enough statistical information to learn about. In this case, VOM models will be equivalent in performance to fixed order Markov models. On the other hand, if the model for a given goal is trained using short sequences and the model for another goal is trained with longer sequences containing the same symbols, the first model will be predicted after observing the same sequence of actions. This happens because the probability mass is distributed among fewer contexts and the corresponding goal will be predicted with higher values. A similar problem occurs with the EMA computation. If sequences are too short, the memory of the model will not be activated and using a smoothing constant $\lambda = 1.0$ (that is equivalent to not computing the EMA and using only the last observation for prediction) can lead to better prediction results.

In future work, we plan to explore techniques in the field of Human Computer Interaction to define the way the user will interact with the agent to provide the examples needed for the model construction. In previous research (Armentano et al. 2006), we found that the interaction with an interface agent generates a greater perception of the computer as a collaborative decision maker and increases the user demand for high quality results. Even though the representation of agents in the user interface increases the level of engagement of users with the application, such representation can lead to misinterpretations about agent actions and expected results; therefore, a careful design of the agent-user interaction is required.

References

- Armentano, M., Amandi, A.: Recognition of user intentions for interface agents with variable order Markov models. In: Houben, G.-J. et al. (eds.) UMAP 2009. Lecture Notes in Computer Science, vol. 5535, pp. 173–184. Springer, Heidelberg (2009)
- Armentano, M.G., Amandi, A.A.: Personalized detection of user intentions. *Knowl. Based Syst.* **24**(8), 1169–1180 (2011)
- Armentano, M., Godoy, D., Amandi, A.: Personal assistants: direct manipulation vs. mixed initiative interfaces. *Int. J. Hum. Comput. Stud.* **64**(1), 27–35 (2006)
- Bauer, M.: Acquisition of abstract plan descriptions for plan recognition. In: AAAI '98/IAAI '98: Proceedings of the Fifteenth National/Tenth Conference on Artificial Intelligence/Innovative Applications of Artificial Intelligence, Menlo Park, pp. 936–941. American Association for Artificial Intelligence, Menlo Park (1998)
- Bauer M.: From interaction data to plan libraries: a clustering approach. In: IJCAI '99: Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence, San Francisco, pp. 962–967. Morgan Kaufmann, San Francisco (1999)
- Begleiter, R., El-yaniv, R., Yona, G.: On prediction using variable order Markov models. *J. Artif. Intell. Res.* **22**, 385–421 (2004)
- Bejerano, G., Yona, G.: Variations on probabilistic suffix trees: statistical modeling and prediction of protein families. *Bioinformatics* **1**(17), 23–43 (2001)

- Blaylock, N.: Towards Tractable Agent-Based Dialogue. Ur csd/ tr880. Computer Science Department, University of Rochester, Rochester (2005)
- Blaylock, N., Allen, J.: Corpus-based, statistical goal recognition. In: Proceedings of the Eighteenth International Joint Conference on Artificial Intelligence, Acapulco, pp. 1303–1308 (2003)
- Blaylock, N., Allen, J.: Recognizing instantiated goals using statistical methods. In: Kaminka, G. (ed.) IJCAI Workshop on Modeling Others from Observations (MOO-2005), Edinburgh, pp. 79–86 (2005)
- Brand, M., Oliver, N., Pentland, A.: Coupled hidden Markov models for complex action recognition. In: Proceedings of the 1997 Conference on Computer Vision and Pattern Recognition (CVPR '97), Washington, pp. 994–999 (1997)
- Bratko, A., Cormack, G.V., Filipic, B., Lynam, T.R., Zupan, B.: Spam filtering using statistical data compression models. *J. Mach. Learn. Res.* **7**, 2673–2698 (2006)
- Brown, S.M.: A decision theoretic approach for interface agent development. PhD thesis, Wright Patterson AFB. AAI9905125 (1998)
- Bui, H., Venkatesh, S., West, G.: Policy recognition in the abstract hidden Markov model. *J. Artif. Intell. Res.* **17**, 451–499 (2002)
- Bui, H.H: A general model for online probabilistic plan recognition. In: Gottlob, G., Walsh, T. (eds.) Proceedings of the Eighteenth International Joint Conference on Artificial Intelligence (IJCAI '03), pp. 1309–1318. Morgan Kaufmann, Acapulco (2003)
- Buntine, W.: Theory refinement on bayesian networks. In: Proceedings of the Seventh Conference on Uncertainty in Artificial Intelligence. Morgan Kaufmann, San Francisco, pp. 52–60 (1991)
- Card, S.K., Newell, A., Moran, T.P.: The Psychology of Human–Computer Interaction. Lawrence Erlbaum, Hillsdale (1983)
- Charniak E., Goldman R.: A probabilistic model of plan recognition. In: Proceedings of the Ninth National Conference on Artificial Intelligence, AAAI'91, vol. 1, pp. 160–165. AAAI Press, Menlo Park (1991)
- Cooper, G., Herskovits, E.: A bayesian method for the induction of probabilistic networks from data. *Mach. Learn.* **9**(4), 309–347 (1992)
- Davison, B.D., Hirsh, H.: Predicting sequences of user actions. In: Predicting the Future: AI Approaches to Time Series. AAAI Press, Menlo Park (1998)
- Dempster, A., Laird, N., Rubin, D.: Maximum likelihood from incomplete data via the em algorithm. *J. Roy. Stat. Soc.* **39**(1), 1–38 (1977)
- Duong, T.V., Phung, D.Q., Bui, H.H., Venkatesh, S.: Human behavior recognition with generic exponential family duration modeling in the hidden semi-Markov model. In: International Conference on Pattern Recognition, Hong Kong, vol. 3, pp. 202–207 (2006)
- Fine, S., Singer, Y., Tishby, N.: The hierarchical hidden Markov model: analysis and applications. *Mach. Learn.* **32**, 41–62 (1998)
- Galata, A., Johnson, N., Hogg, D.: Learning variable-length Markov models of behavior. *Comput. Vis. Image Underst.* **81**(3), 398–413 (2001)
- Garland, A., Lesh, N.: Learning hierarchical task models by demonstration, Technical Report TR2002-04. Mitsubishi Electric Research Laboratories, Cambridge (2002)
- Geib, C.W., Maraist, J., Goldman, R.P.: A new probabilistic plan recognition algorithm based on string rewriting. In: Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS 2008), Sydney, pp. 91–98 (2008)
- Goldman, R., Geib, C., Miller, C.: Learning hierarchical task models by defining and refining examples. In: Proceedings of the 15th Annual Conference on Uncertainty in Artificial Intelligence (UAI-99), pp. 245–254. Morgan Kaufmann, San Francisco (1999)
- Gorniak, P., Poole, D.: Building a stochastic dynamic model of application. In: Boutilier, C., Goldszmidt, M. (eds.) Sixteenth Conference on Uncertainty in Artificial Intelligence (UAI-2000), pp. 230–237. Stanford University, Morgan Kaufmann, San Francisco (2000)
- Hong, J.: Goal recognition through goal graph analysis. *J. Artif. Intell. Res.* **15**, 1–30 (2001)
- Horvitz, E., Breese, J., Heckerman, D., Hovel, D., Rommelse, K.: The Lumière project: Bayesian user modeling for inferring the goals and needs of software users. In: Cooper, G.F., Moral, S. (eds.) Proceedings of the Fourteenth Conference on Uncertainty in Artificial Intelligence, pp. 256–265. Morgan Kaufmann, San Mateo (1998)
- Hu, J., Turin, W., Brown, M.K.: Language modeling using stochastic automata with variable length contexts. *Comput. Speech Lang.* **11**(1), 1–16 (1997)
- Hunter, J.S.: The exponentially weighted moving average. *J. Qual. Technol.* **18**(4), 203–209 (1986)

- Kautz, H.: A formal theory of plan recognition and its implementation. In: Allen, J.F., Kautz, H.A., Pelavin, R., Tenenber, J. (eds.) Reasoning About Plans, pp. 69–125. Morgan Kaufmann, San Mateo (1991)
- Kearns, M., Mansour, Y., Ron D., Rubinfeld, R., Schapire, R.E., Sellie, L.: On the learnability of discrete distributions. In: Proceedings of the 26th Annual ACM Symposium on Theory of Computing, pp. 273–282. ACM Press, New York (1994)
- Köck, M., Paramythis, A.: Activity sequence modelling and dynamic clustering for personalized e-learning. *User Model. User Adapt. Interact.* **21**, 51–97 (2011)
- Lesh, N.: Scalable and adaptive goal recognition. PhD thesis, University of Washington, Washington (1998)
- Lesh, N., Rich, C., Sidner, C.L.: Using plan recognition in human-computer collaboration. In: Proceedings of the Seventh International Conference on User modeling, Secaucus, pp. 23–32. Springer, New York (1999)
- Liao, L., Patterson, D.J., Fox, D., Kautz, H.A.: Learning and inferring transportation routines. *Artifi. Intell.* **171**(5–6), 311–331 (2007)
- Lieberman, H.: *Your Wish Is My Command: Programming by Example*. Morgan Kaufmann, San Francisco (2001)
- Madani, O., Bui, H., Yeh, E.: Efficient online learning and prediction of user's desktop activities. In: Proceedings of IJCAI 2009, Pasadena, vol. 3, pp. 1457–1462 (2009)
- Maes, P.: Agents that reduce work and information overload. *Commun. ACM* **37**, 30–40 (1994)
- Nguyen, N.T., Phung, D.Q., Venkatesh, S., Bui, H.H.: Learning and detecting activities from movement trajectories using the hierarchical hidden Markov model. In: IEEE Computer Vision and Pattern Recognition or CVPR, pp. 955–960. IEEE Computer Society, Washington (2005)
- Nkambou, R., Fournier-Viger, P., Nguifo, E.M.: Learning task models in ill-defined domain using an hybrid knowledge discovery framework. *Knowl. Based Syst.* **24**, 176–185 (2011)
- Nunez-Yanez, J.L., Chouliaras, V.A.: A configurable statistical lossless compression core based on variable order Markov modeling and arithmetic coding. *IEEE Trans. Comput. Comput.* **54**(11), 1345–1359 (2005)
- Oliver, N., Horvitz, E., Garg, A.: Layered representations for human activity recognition. In Proceedings of the 4th IEEE International Conference on Multimodal Interfaces (ICMI 2002), pp. 3–8. IEEE Computer Society, Washington (2002)
- Pascal, J.H., Poupart, P., Boutilier, C., Mihailidis, A.: Semi-supervised learning of a POMDP model of patient-caregiver interactions. In: Proceedings of IJCAI Workshop on Modeling Others from Observations, Edinburgh, pp. 101–110 (2005)
- Philipose, M., Fishkin, K.P., Perkowitz, M., Patterson, D.J., Fox, D., Kautz, H., Hahnel, D.: Inferring activities from interactions with objects. *Pervasive Comput. Mag.* **3**(4), 10–17 (2004)
- Pynadath D.V., Wellman M.P.: Probabilistic state-dependent grammars for plan recognition. In Proceedings of the 16th Conference on Uncertainty in Artificial Intelligence (UAI '00), Stanford, pp. 507–514 (2000)
- Rabin, M.O.: Probabilistic automata. *Inform. Control* **6**(3), 230–245 (1963)
- Rich, C., Sidner, C.L., Lesh, N.: COLLAGEN: applying collaborative discourse theory to human-computer interaction. *AI Mag.* **22**(4), 15–26 (2001)
- Rissanen, J.: A universal data compression system. *IEEE Trans. Inform. Theory* **29**(5), 656–663 (1983)
- Ron, D., Singer, Y., Tishby, N.: The power of amnesia: learning probabilistic automata with variable memory length. *Mach. Learn.* **25**(2–3), 117–149 (1996)
- Shmilovici, A., Ben-Gal, I.: Using a VOM model for reconstructing potential coding regions in est sequences. *Comput. Stat.* **22**(1), 49–69 (2007)
- Soller A.: Adaptive support for distributed collaboration. In: Brusilovsky, P., Kobsa, A., Neijl, W. (eds.) *The Adaptive Web*, pp. 507–514. Springer, Berlin (2007)
- Spirtes, P., Glymour, C., Scheines, R.: *Causation, Prediction, and Search*. 2nd edn. The MIT Press, Cambridge (2000)
- Stumpf, S., Bao, X., Dragunov, A., Dietterich, T.G., Herlocker, J., Johnsrude, K., Li, L., Shen, J.: Predicting user tasks: I know what you're doing! In: 20th National Conference on Artificial Intelligence (AAAI-05), Workshop on Human Comprehensible Machine Learning, Pittsburgh (2005)
- Whitworth, B.: Polite computing. *Behav. Inform. Technol.* **24**(5), 353–363 (2005)

Author Biographies

Marcelo G. Armentano is an Assistant Professor of Computer Science at UNICEN University, working in the area of Intelligent Agents and Knowledge Management. He received a Systems Engineer degree at UNICEN University in 2003 and a Master in Systems Engineer degree at UNICEN University in 2006. He completed his Ph.D. in 2006 at UNICEN University, under the supervision of Analía Amandi, in the area of Intelligent Agents. His research interests include intelligent agents, knowledge management, user modeling, adaptation and personalization.

Analía A. Amandi is a Full Professor of Computer Science and head of the Intelligent Agents Group at ISISTAN Research Institute at UNICEN University in Tandil, Argentina. She received a Ph.D. in Computer Science in the UFRGS, Porto Alegre, Brazil. Her research interests include intelligent agents and knowledge management.